TEACHING WALLAH

Teaching Exam

Subject – Computer Science

Topic – Digital Logic

Lecture No.- 5

By- Rahul Sir

Topics to be Covered

1:- Binary Multiplication
2:- Binary Division
3:- Logic Gates
4:- Universal Logic Gates
5:- How to create basic Logic Gate using Universal Gate
6:- Boolean Algebra
7:- Laws of Boolean Algebra
8:- Conclusion on Boolean Algebra

Topic : BINARY MULTIPLICATION

TEACHING WALLAH

Binary Multiplication is performed in the same manner as decimal numbers are multiplied. However, there are some specific rules regarding the multiplication among the binary digits 0 and 1, which we need to follow while performing the division of Binary Multiplication. Binary Multiplication rules are shown in the Binary Multiplication Table below:

Table for Binary Multiplication Rule

Rules for Multiplication	Meaning
$0 \times 0 = 0$	If 0 (zero) is multiplied by another 0 (zero), then the result is 0 (zero).
$0 \times 1 = 0$	If 0 (zero) is multiplied by 1 (one), then the result is 0 (zero).
$1 \times 0 = 0$	If 1 (one) is multiplied by 0 (zero), then the result is 0 (zero).
$1 \times 1 = 1$	If 1 (one) is multiplied by another 1 (one), then the result is 1 (one).

How to do Binary Multiplication?

There are five key steps involved in Binary Multiplication:,

Step 1: Write the multiplicand and the multiplier one below the other, aligning the rightmost digits.

Step 2: Multiply the multiplicand by each digit of the multiplier, starting from the rightmost digit.

Step 3: Move to the next digit of the multiplier and multiply it by the multiplicand. Write the result below the multiplier after shifting it to the left by one position.

Step 4: Repeat this process for each digit of the multiplier, shifting the result to the left by one more position each time.

Step 5: Add all the results using binary addition rules. The final sum is the product of the two binary numbers.

You can also use the Binary Multiplication Calculator to easily calculate the multiplication of two binary numbers.

How to do Binary Multiplication?

There are five key steps involved in Binary Multiplication:,

Step 1: Write the multiplicand and the multiplier one below the other, aligning the rightmost digits.

Step 2: Multiply the multiplicand by each digit of the multiplier, starting from the rightmost digit.

Step 3: Move to the next digit of the multiplier and multiply it by the multiplicand. Write the result below the multiplier after shifting it to the left by one position.

Step 4: Repeat this process for each digit of the multiplier, shifting the result to the left by one more position each time.

Step 5: Add all the results using binary addition rules. The final sum is the product of the two binary numbers.

You can also use the Binary Multiplication Calculator to easily calculate the multiplication of two binary numbers.

Topic : BINARY MULTIPLICATION

TEACHING WALLAH

Question 1: $(1010)_2 \times (101)_2$

Step 1: Write the multiplicand $(1010)_2$ and the multiplier $(101)_2$ one below the other, aligning the rightmost digits.

Step 2: Multiply the multiplicand by each digit of the multiplier, starting from the rightmost digit. The rightmost digit of the multiplier is 1, so we write down the multiplicand as it is.

Step 3: Move to the next digit of the multiplier and multiply it by the multiplicand. The next digit of the multiplier is 0, so we write down 0 after shifting it to the left by one position.

Step 4: Move to the next digit of the multiplier and multiply it by the multiplicand. The next digit of the multiplier is 1, so we write down the multiplicand after shifting it to the left by two positions.

Step 5: Add all the results using binary addition rules. The final sum $(110010)_2$ is the product of the two binary numbers.

```
   1010
  x 101
  ----
   1010
  0000 x
 1010 x x
 ------
 110010
```

Ans – $(110010)_2$

```
    1 0 1 0
  x   1 0 1
  ---------
    1 0 1 0
  0 0 0 0 *
+ 1 0 1 0 * *
  ---------
  1 1 0 0 1 0
```

$(1010)_2 \times (101)_2 = (110010)_2$

So, the product of $(1010)_2 \times (101)_2$ is $(110010)_2$

Question 3: $(0.101)_2 \times (0.11)_2$

Step 1: Write the multiplicand $(0.101)_2$ and the multiplier $(0.11)_2$ one below the other, aligning the rightmost digits.

Step 2: Multiply the multiplicand by each digit of the multiplier, starting from the rightmost digit. The rightmost digit of the multiplier is 1, so we write down the multiplicand as it is.

Step 3: Move to the next digit of the multiplier and multiply it by the multiplicand. The next digit of the multiplier is 1, so we write down the multiplicand after shifting it to the left by one position.

Step 4: Move to the next digit of the multiplier and multiply it by the multiplicand. The next digit of the multiplier is 0, so we write down 0 after shifting it to the left by two positions.

Step 5: Add all the results using binary addition rules. The final sum $(001111)_2$ is the product of the two binary numbers.

Step 6: Count the total number of digits (d) after the decimal point in both the multiplicands and the multiplier. Place the decimal point in the product after (d) digits from right side.

```
  0.101
 x 0.11
 ------
   0101
  0101 x
 0000 x x
 ------
 0.01111
```

```
   0 . 1 0 1
 x   0 . 1 1
 -----------
       1 0 1
     1 0 1 *
 + 0 0 0 0 * *
 -----------
   0 0 1 1 1 1
 = 0 . 0 1 1 1 1
```

$(0.101)_2 \times (0.11)_2 = (0.01111)_2$

So, the product of $(0.101)2 \times (0.11)_2$ is $(0.01111)_2$

Topic : BINARY DIVISON

TEACHING WALLAH

Binary division is a mathematical operation that involves dividing two binary numbers, which are numbers composed of only 0's and 1's. Binary division is similar to decimal division, except that the base of the number system is 2 instead of 10.

Rules for Binary Division

$\frac{100000}{0}$, $\frac{5}{0}$ $\frac{0}{0}$, $\frac{\text{Some value}}{0}$ = undefine ⇒ ∞

A ÷ B	Result
0 ÷ 0	∞
0 ÷ 1	0
1 ÷ 1	1
1 ÷ 0	∞

Topic : BINARY DIVISON

TEACHING WALLAH

Example 1: $(1011011)_2 \div (101)_2$

We start by taking the first three digits of the dividend $(101)_2$ which is equal to the divisor.

Step 1: *Write 1 as the first digit of the quotient. Then, we subtract the divisor from the first part of the dividend and write down the remainder.*

Step 2: *Next, we bring down the next digit of the dividend (1). Now we have $(1)_2$ which is less than the divisor $(101)_2$. So, we write 0 in the quotient.*

Step 3: *Next, we bring down the next digit of the dividend (0). Now we have $(10)_2$ which is less than the divisor $(101)_2$. So, we write 0 in the quotient.*

Step 4: *Next, we bring down the next digit of the dividend (1). Now we have $(101)_2$ which is equal to the divisor $(101)_2$. So, we write 1 in the quotient. We subtract the divisor from the current part of the dividend and write down the remainder.*

Step 5: *Finally, we bring down the last digit of the dividend (1). Now we have $(1)_2$ which is less than the divisor $(101)_2$. So, we write 0 in the quotient and 1 as the remainder.*

1001

101) 1011011

101

000101

101

000001 → Remainder

Result $(1001)_2$ and Remainder $(01)_2$

10010

101) 1011011

- 101

0101

- 101

01

If This Value is always Greater or Equal then it gives quotient as 1

If It is less then take quotient as 0 till it comes to equal or greater

$(1011011)_2 \div (101)_2 = (10010)_2$

So, the quotient of $(1011011)_2 \div (101)_2$ is $(10010)_2$ and the remainder is $(1)_2$

EXAMPLE 2:- $(1010011.1010)_2 \div (100)_2$

Step 1: Write 1 as the first digit of the quotient. Then, we subtract the divisor from the first part of the dividend and write down the remainder.

Step 2: Next, we bring down the next digit of the dividend (0). Now we have $(10)_2$ which is less than the divisor $(100)_2$. So, we write 0 in the quotient.

Step 3: Next, we bring down the next digit of the dividend (0). Now we have $(100)_2$ which is equal to the divisor $(100)_2$. So, we write 1 in the quotient. We subtract the divisor from the current part of the dividend and write down the remainder.

Step 4: Next, we bring down the next digit of the dividend (1). Now we have $(1)_2$ which is less than the divisor $(100)_2$. So, we write 0 in the quotient.

Step 5: Next, we bring down the next digit of the dividend (1). Now we have $(11)_2$ which is less than the divisor $(100)_2$. So, we write 0 in the quotient.

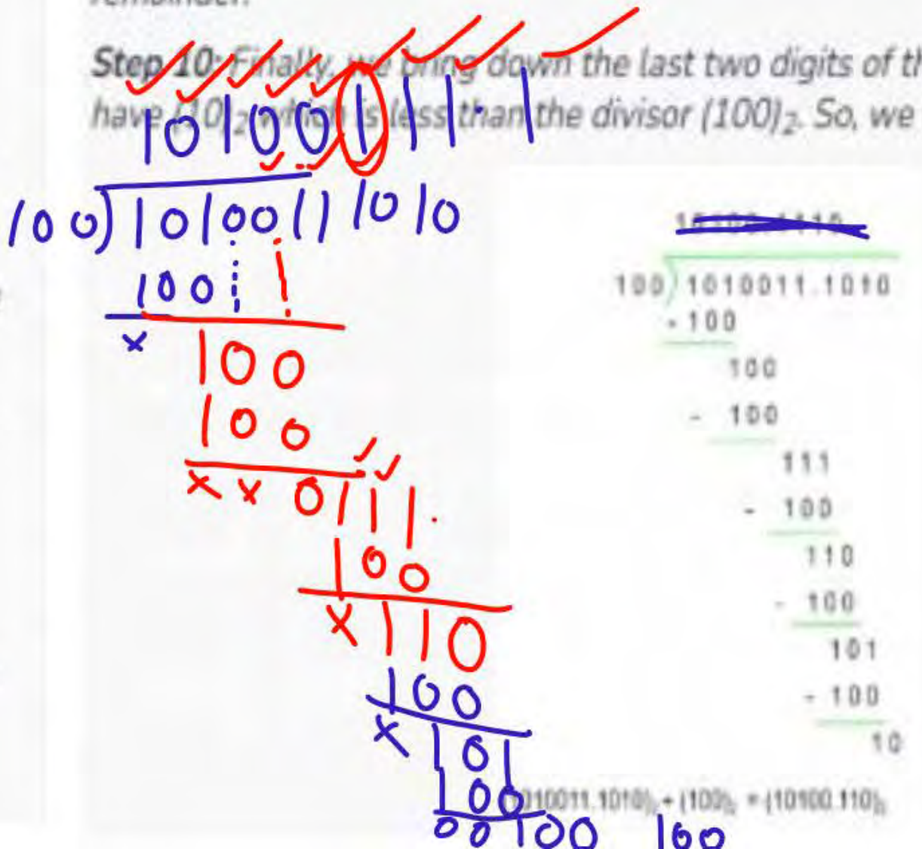
Step 6: Next, we bring down the next digit of the dividend (.). This indicates that we are now moving into the fractional part of the division. We continue the process as before.

Step 7: Next, we bring down the next digit of the dividend (1). Now we have $(111)_2$ which is greater than the divisor $(100)_2$. So, we write 1 in the quotient. We subtract the divisor from the current part of the dividend and write down the remainder.

Step 8: Next, we bring down the next digit of the dividend (0). Now we have $(110)_2$ which is greater than the divisor $(100)_2$. So, we write 1 in the quotient. We subtract the divisor from the current part of the dividend and write down the remainder.

Step 9: Next, we bring down the next digit of the dividend (1). Now we have $(101)_2$ which is equal to the divisor $(100)_2$. So, we write 1 in the quotient. We subtract the divisor from the current part of the dividend and write down the remainder.

Step 10: Finally, we bring down the last two digits of the dividend (0). Now we have $(10)_2$ which is less than the divisor $(100)_2$. So, we write it as the remainder.



Topic : PRACTICE ON BINARY DIVISON

TEACHING WALLAH

Q1. Divide $(10110)_2$ by $(10)_2$ H.W

Q2. Is $(10010101)_2$ is a multiple of $(11)_2$?

Q3. Divide $(11001110)_2$ by $(1001)_2$

Q4. Divide $(11110010)_2$ by $(1010)_2$

Q5. Divide $(11010)_2$ by $(101)_2$

Q6. $(10011001)_2 \div (1001)_2$

1001) 10011001 (10001

1001

xxxx

1001

1001

xxxx

11000.00111

1010) 11110010

1010

x 1010

1010

xxxx010000

1010

10110

1010

x 1100

1010

0010

11000.00111...

What is a Logic Gate?

A gate can be defined as a digital circuit that can allow a signal (electric current) to pass or stop. A type of gate that allows a signal to pass through when certain logical conditions are met. Different logic gates have different logical conditions.

Truth Table: A truth table is a table that shows all possible combinations of inputs and outputs for a logic gate.

Types of Gate

Given Below are the different types of Logic Gates :

- AND Gate(.)
- OR Gate(+)
- NOT Gate(')
- XOR Gate
- NAND Gate
- NOR Gate
- XNOR Gate
- Buffer Gate
- Universal Logic Gates

TEACHING WALLAH

AND Gate(.)

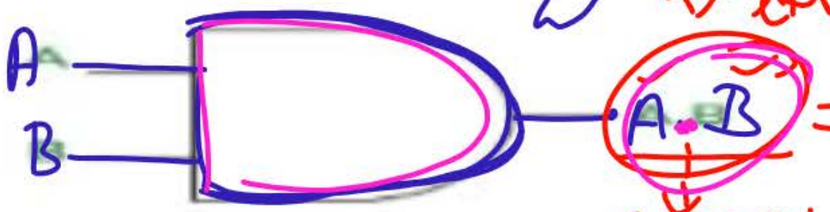
The AND gate gives an output of 1 when if both the two inputs are 1, it gives 0 otherwise. For n-input gate if all the inputs are 1 then 1 otherwise 0.The AND gate operation is similar to the standard multiplication of 1s and 0s.The (.) dot represents the AND operation.

A	B	C		Y = A.B.C
0	0	0	=	0
0	0	1	=	0
0	1	0	=	0
0	1	1	=	0
1	0	0	=	0
1	0	1	=	0
1	1	0	=	0
1	1	1	=	1

Series combination

$Y = A \cdot B \cdot C$

2- Input AND Gate



Boolean expression

$= Y \Rightarrow$ output 1

AND Logic

X=(A.B)

Total possibility of Input $\Rightarrow 2^n \Rightarrow 2^2 \Rightarrow 4$

Where n = no of Input = 2

Where,

X is the output of gate

A and B are the inputs of the gate

Condition

Note:- In AND Gate all the input should be high to get logic ON at output side

Truth Table

A (Input 1)	B (Input 2)	X = (A.B)
0	0	0
0	1	0
1	0	0
1	1	1

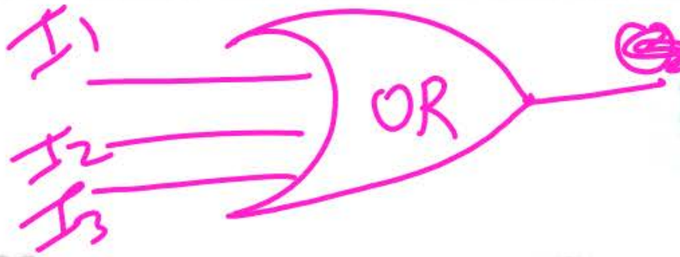
AND Gate

TEACHING WALLAH

A B C = A+B+C = Y
0 0 0 = 0
0 0 1 = 1
0 1 0 = 1
0 1 1 = 1
1 0 0 = 1
1 0 1 = 1
1 1 0 = 1
1 1 1 = 1

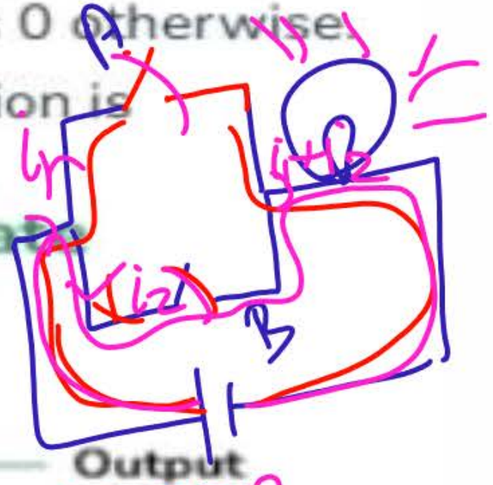
OR Gate(+)

The OR gate gives an output of 1 if either of the two inputs are 1, it gives 0 otherwise. For n-input gate if all the inputs are 0 then 0 otherwise 1.The OR Operation is represented by the +.



Y = A+B+C

2-Input OR Gate



Y = A+B

The Expression for OR Gate is given as

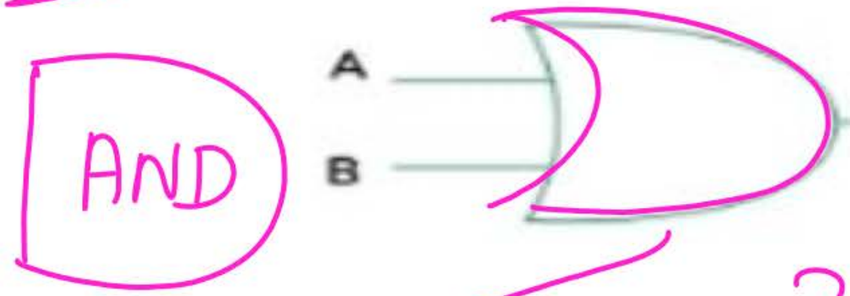
$2^n = 2^3 = 8$

$X=A+B$

Where,

X is the output of gate

A and B are the inputs of the gate



$2^2 = 4$

Truth Table

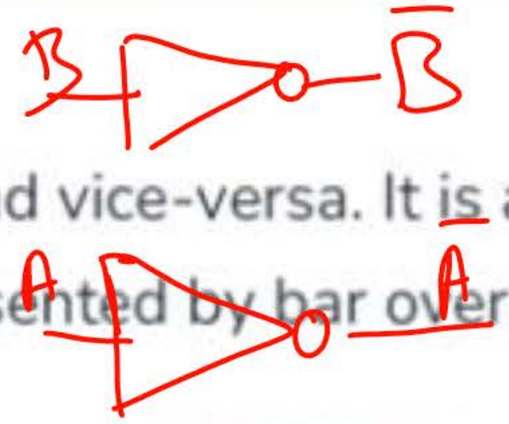
Y = A+B

Input A	Input B	Output
0	0	0
0	1	1
1	0	1
1	1	1

Note :- If Any One input is High or Both High then output Should be logic ON

NOT Gate(')

The NOT gate gives an output of 1 if the input is 0 and vice-versa. It is also known as Inverters. In Boolean algebra NOT operation is represented by bar over the variable such as $\overline{A}$.



NOT Gate

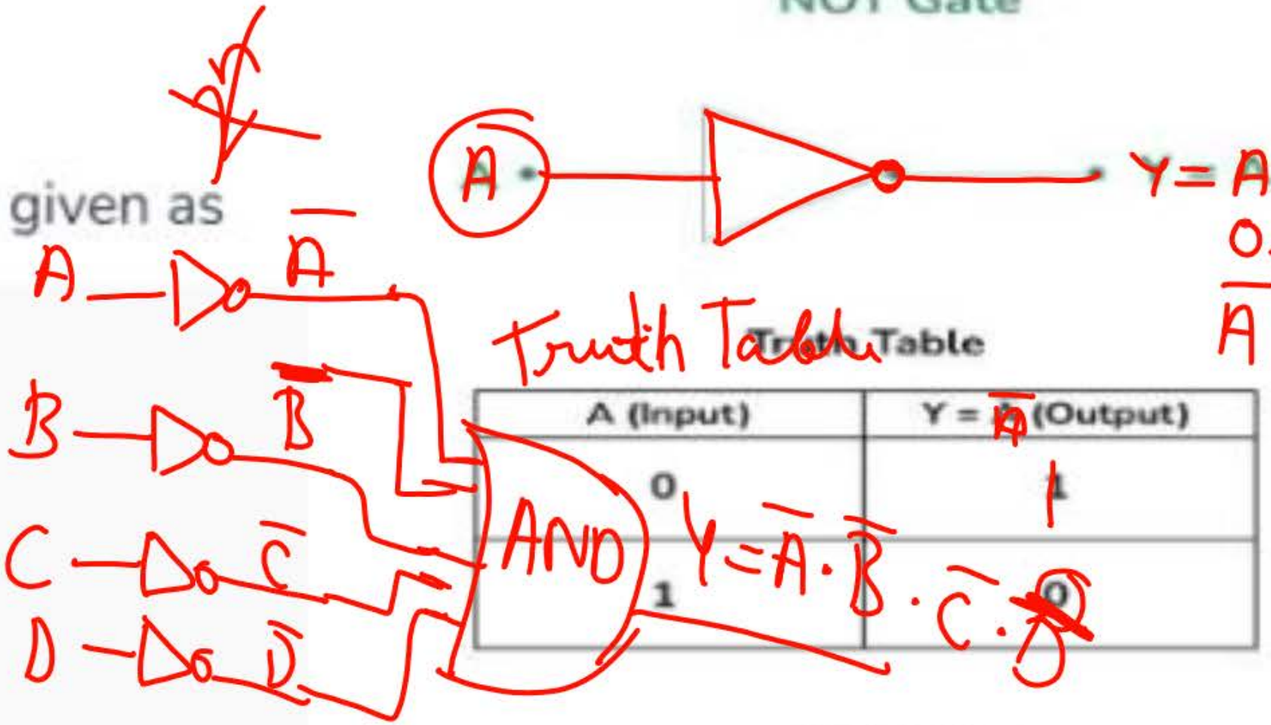
The Expression for NOT Gate can be given as

$$Y = \overline{A}$$

Where,

X is the output of gate

A is the inputs of the gate



Truth Table

A (Input)	Y = $\overline{A}$ (Output)
0	1
1	0

NOT Gate

TEACHING WALLAH

XOR Gate

The XOR gate gives an output of 1 if either both inputs are different, it gives 0 if they are same. For n-input gate if the number of input 1 are odd then it gives 1 otherwise 0.For a two-input XOR gate it means that the output is true only if exactly one of the inputs is true.

$Y = A\bar{B}C + ABC + B\bar{A}C + BA\bar{C} + C\bar{A}B + CA\bar{B}$

The Expression for XOR Gate can be given as

$X = A'B + AB'$

Where,

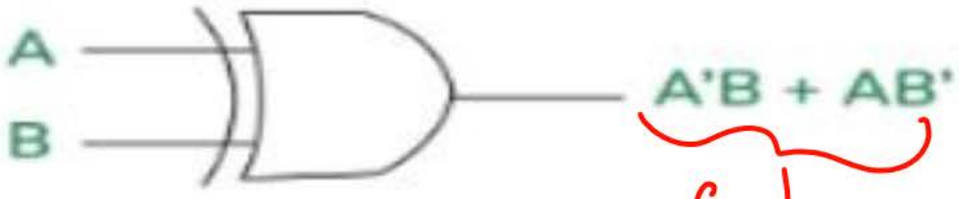
X is the output

A and B are the inputs

Note:- If all the input is low or high the Output should be low (logic off)

A	B	C	Y
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	0

XOR Gate



final result ⇒ Y

Truth Table

A (Input 1)	B (Input 2)	X = A'B + AB'
0	0	0
0	1	1
1	0	1
1	1	0

XOR Gate

The XOR gate gives an output of 1 if either both inputs are different, it gives 0 if they are same. For n-input gate if the number of input 1 are odd then it gives 1 otherwise 0.For a two-input XOR gate it means that the output is true only if exactly one of the inputs is true.

The Expression for XOR Gate can be given as

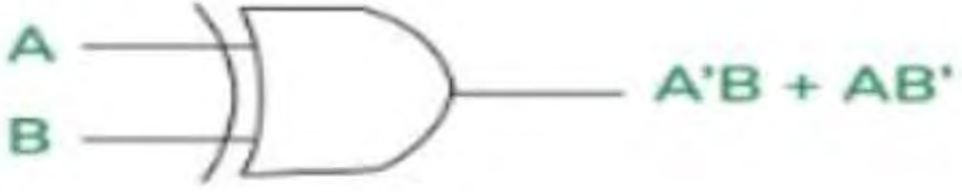
$$X = A'B + AB'$$

Where,

X is the output

A and B are the inputs

XOR Gate



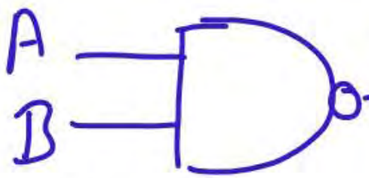
Truth Table

A (Input 1)	B (Input 2)	X = A'B + AB'
0	0	0
0	1	1
1	0	1
1	1	0

TEACHING WALLAH

NAND Gate (is opposite to AND Gate)

The NAND gate (negated AND) gives an output of 0 if both inputs are 1, it gives 1 otherwise. For n-input gate if all inputs are 1 then it gives 0 otherwise 1.The Term "NAND" can be said as "Not AND".



$Y = \overline{(A \cdot B)}$

or

$(A \cdot B)'$

2-Input NAND Gate

$Y = \overline{A+B} \Leftrightarrow \overline{A \cdot B}$

The Expression for NAND Gate can be given as

Special Note

$$Y = \overline{A.B}$$

NAND

WHEN BOTH INPUT HIGH THE OUTPUT IS LOW

Where,

Y is the Output of gate

A and B is the input of gate

Truth Table

Input A	Input B	X = (A.B)'
0	0	1
0	1	1
1	0	1
1	1	0

Note:- In NAND Gate if Both input is High the output is low and if any input is High then output is High

NAND Gate

TEACHING WALLAH

NOR Gate (is opposite to OR Gate)

The NOR gate (negated OR) gives an output of 1 only if both inputs are 0, it gives 0 otherwise. For n-input gate if all inputs are 0 then it gives 1 otherwise 0.The "NOR" can be said as "NOT OR".

Special Note

The Expression for NOR Gate can be given as

$$Y = \overline{A + B}$$

Where,

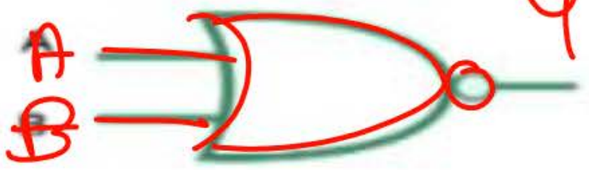
Y is the Output of gate

A and B is the input of gate

NOR

WHEN BOTH INPUT LOW
THE OUTPUT IS HIGH

2- Input NOR Gate



$Y = \overline{(A+B)} \Leftrightarrow \overline{A} \cdot \overline{B}$

or

$(A+B)'$

Truth Table

Input A	Input B	0 = (A + B)'
0	0	1
0	1	0
1	0	0
1	1	0

NOR Gate

Note:- If all the input is low the output is high but when Any of Input High the output is low (logic off)

TEACHING WALLAH

XNOR Gate

The XNOR gate (negated XOR) gives an output of 1 if both inputs are same and 0 if they are different. For n-input gate if the number of input 1 are even then it gives 1 otherwise 0 or if the number of input 0 is even then the output is 1, otherwise 0'. The "XNOR" can be said as "Exclusive NOR".

The Expression for XNOR Gate can be given as

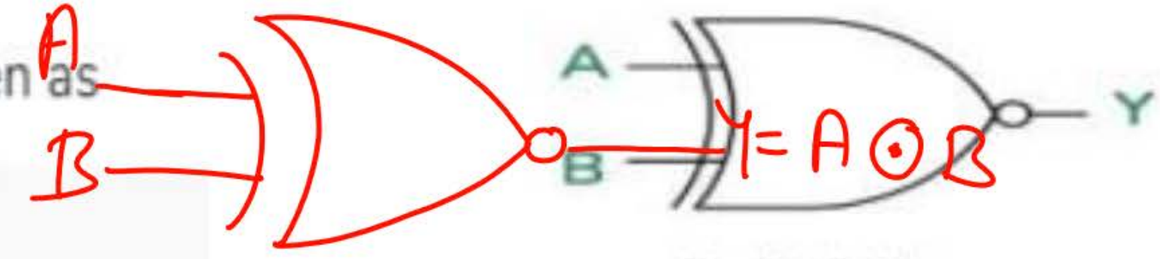
$Y = A \odot B$

Where,

Y is the Output of gate

A and B is the input of gate

Note:- If all the Input is low or high then Output will be High (logic ON)



What is XNOR Gate?

Truth Table

Input A	Input B	Output
0	0	1
0	1	0
1	0	0
1	1	1

XNOR Gate

Buffer Gate

A Buffer Gate is a digital logic gate that amplifies a signal. It doesn't change the logic state (it outputs exactly what is input), but its primary purpose is to ensure that the signal can drive more circuits without weakening. Essentially, a buffer isolates the input from the output, making the output more robust.

- If the input is **1** , the buffer outputs **1** .
- If the input is **0** , the buffer outputs **0** .

Buffers are often used when a signal needs to drive a large number of other gates or when it needs to travel over a long distance without losing strength.

The Expression for Buffer can be given as

$Y=A$

Where,

Y is the Output of gate

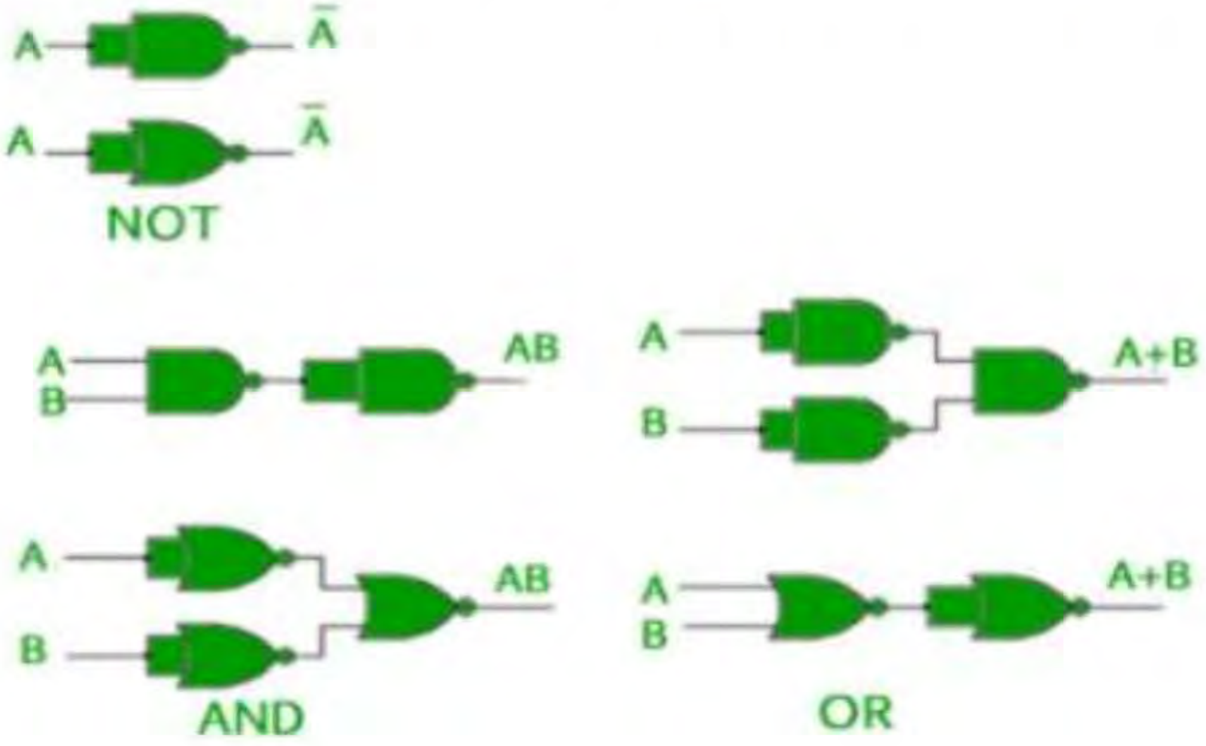
A is the input of gate

Truth Table

Input(A)	Output(Y)
0	0
1	1

Universal Logic Gates

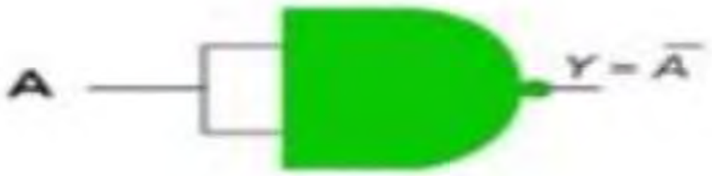
Out of the eight logic gates discussed above, NAND and NOR are also known as **universal gates** since they can be used to implement any digital circuit without using any other gate. This means that every gate can be created by NAND or NOR gates only. Implementation of three basic gates using NAND and NOR gates is shown below -



NOT Gate (Inverter)

Circuit: Connect both inputs of a NAND gate to the same input signal.

NOT Gate using NAND Gate



Truth Table

A (Input)	B (Input)	$Y = (\overline{AB}) = \overline{A}$
0	0	1
1	1	0

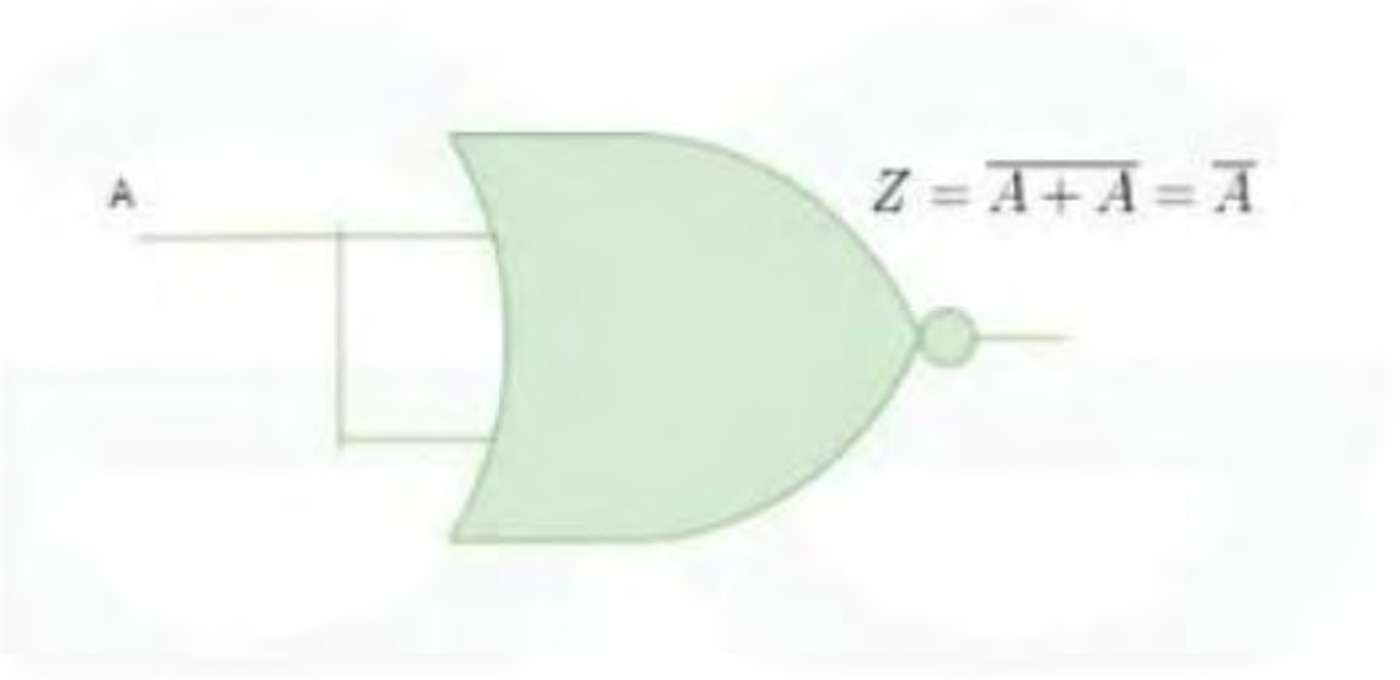
NOT GATE USING NAND GATE

NOT Gate (Inverter)

Circuit: Connect both inputs of a NOR gate to the same input signal.

Truth Table

A	Output (NOT A)
0	1
1	0

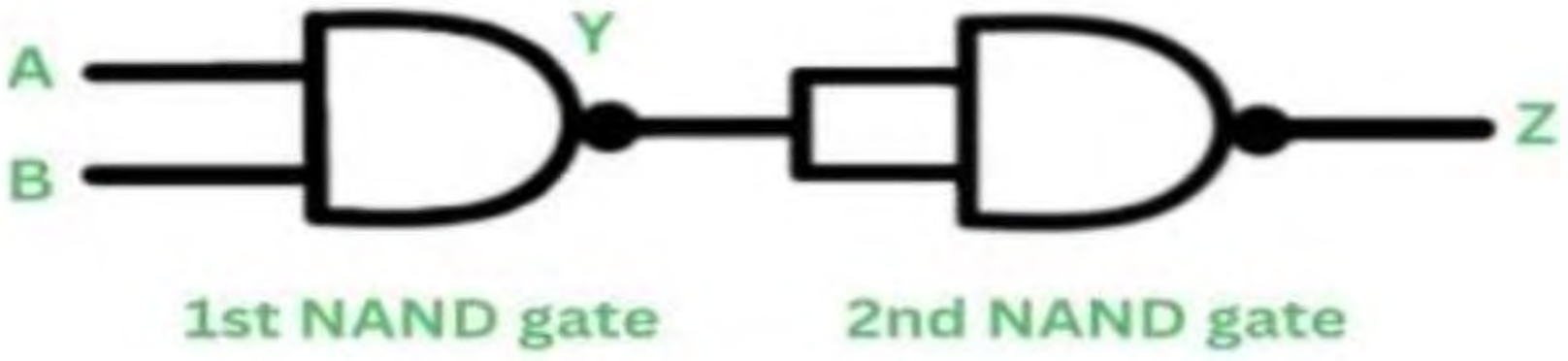


AND Gate

Circuit: Connect the outputs of two NAND gates to the inputs of a third NAND gate.

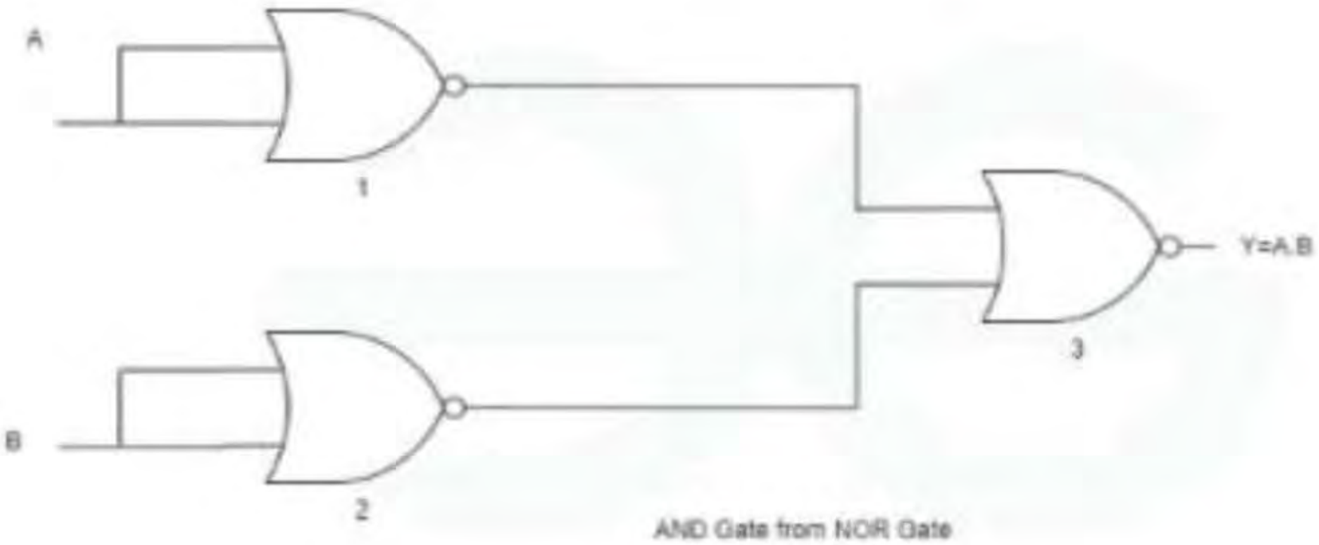
Truth Table

A	B	Z
0	0	0
0	1	0
1	0	0
1	1	1



AND Gate

Circuit: Use three NOR gates. First, connect each input to a NOR gate and then connect their outputs to the inputs of another NOR gate.



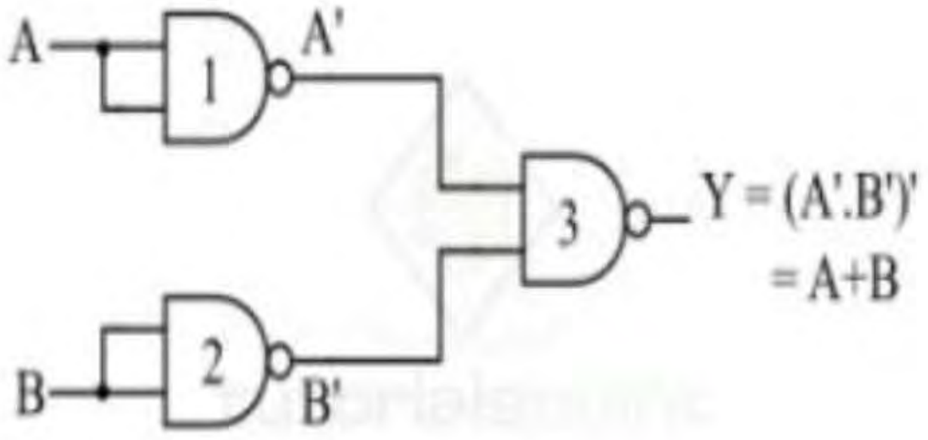
AND Gate from NOR Gate

Truth Table

A	B	Output (A AND B)
0	0	0
0	1	0
1	0	0
1	1	1

TEACHING WALLAH

The NAND gate is a universal gate, therefore, it can be used to realize the OR gate.
The implementation of OR gate using the NAND gate is shown in Figure-3.

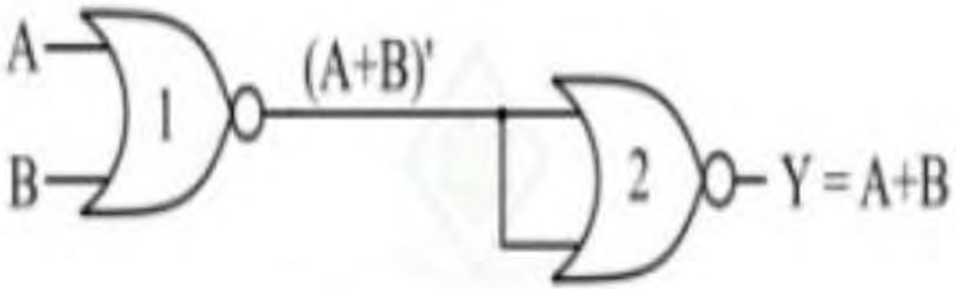


Truth Table

A	B	Output (A OR B)
0	0	0
1	0	1
0	1	1
1	1	1

Implementation of OR Gate from NOR Gate

As we know, the NOR gate is a type of universal logic gate, therefore, using NOR gates only, we can implement the OR operation. The logic diagram of OR Gate using NOR Gate is shown in Figure-3.



Truth Table

A	B	A NOR B
0	0	1
0	1	0
1	0	0
1	1	0

Fundamental Laws & Theorems of Boolean Algebra

The fundamental laws and theorems of Boolean algebra simplify logical expressions and help in designing digital circuits. Some of the core laws and theorems in Boolean algebra are:

1. Commutative Law

The Commutative Law states that the order in which two variables are combined using the AND or OR operators does not affect the result. This law allows us to reorder terms without changing the output.

- **Commutative Law for AND:**

$$A \cdot B = B \cdot A$$

Example:

$$1 \cdot 0 = 0 \cdot 1 = 0$$

- **Commutative Law for OR:**

$$A + B = B + A$$

Example:

$$1 + 0 = 0 + 1 = 1$$

2. Associative Law

The Associative Law states that when three or more variables are combined using the AND or OR operators, the grouping of the variables does not affect the result. This law allows us to regroup terms without changing the output.

- **Associative Law for AND**:

$$A \cdot (B \cdot C) = (A \cdot B) \cdot C$$

Example:
$1 \cdot (0 \cdot 1) = (1 \cdot 0) \cdot 1 = 0$

- **Associative Law for OR**:

$$A + (B + C) = (A + B) + C$$

Example:
$1 + (0 + 1) = (1 + 0) + 1 = 1$

3. Distributive Law

The Distributive Law describes how the AND and OR operations distribute over each other. It is similar to how multiplication distributes over addition in arithmetic. This law allows the factoring of Boolean expressions, similar to factoring algebraic expressions.

- **Distributive Law for AND over OR**:

$$A \cdot (B + C) = (A \cdot B) + (A \cdot C)$$

Example:
$1 \cdot (0 + 1) = (1 \cdot 0) + (1 \cdot 1) = 0 + 1 = 1$

- **Distributive Law for OR over AND**:

$$A + (B \cdot C) = (A + B) \cdot (A + C)$$

Example:
$1 + (0 \cdot 1) = (1 + 0) \cdot (1 + 1) = 1 \cdot 1 = 1$

4. Identity Law

The Identity Law states that any variable ANDed with 1 or ORed with 0 will result in the original variable itself. This law shows that the identity elements for AND and OR operations are 1 and 0, respectively.

- **Identity Law for AND**:

$A \cdot 1 = A$

Example:
$1 \cdot 1 = 1$

- **Identity Law for OR**:

$A + 0 = A$

Example:
$1 + 0 = 1$

5. Complement Law

The Complement Law involves the negation of a variable and provides the result when a variable is combined with its complement (opposite). This law shows that a variable ANDed with its complement will always be 0 and a variable ORed with its complement will always be 1.

- **Complement Law for AND**:

$$A \cdot A' = 0$$

Example:
$1 \cdot 1' = 1 \cdot 0 = 0$

- **Complement Law for OR**:

$$A + A' = 1$$

Example:
$1 + 1' = 1 + 0 = 1$

6. Inversion Law

The Inversion Law is a unique principle in Boolean algebra, stating that the complement of the complement of any variable is equal to the variable itself.

$(A') = A$

Example:

$(1')' = (0)' = 1$

6. Inversion Law

The Inversion Law is a unique principle in Boolean algebra, stating that the complement of the complement of any variable is equal to the variable itself.

$(A') = A$

Example:

$(1')' = (0)' = 1$

7. De Morgan's Theorems

De Morgan's Theorems provide a way to simplify expressions involving negations and are very useful in digital circuit design.

De Morgan's First Theorem: The negation of an AND operation is equal to the OR operation of the negations of the operands.

$(A \cdot B)' = A' + B'$

Example:
Expression: $(1 \cdot 0)'$
Simplified: $1' + 0' = 0 + 1 = 1$

De Morgan's Second Theorem: The negation of an OR operation is equal to the AND operation of the negations of the operands.

$(A+B)' = A' \cdot B'$

Example:
Expression: $(1 + 0)'$
Simplified: $1' \cdot 0' = 0 \cdot 1 = 0$

TEACHING WALLAH

Idempotent Law

$A * A = A$

$A + A = A$

Absorption

$A + (A * B) = A$

$A * (A + B) = A$

$(A * B) + (A * \sim B) = A$

$(A + B) * (A + \sim B) = A$

$A + (\sim A * B) = A + B$

$A * (\sim A + B) = A * B$

TEACHING WALLAH

Basic Laws of Boolean Algebra

In all the cases given below A can either be 0 or 1

1. $A \cdot 0 = 0$
2. $A \cdot 1 = A$
3. $A \cdot A = A$
4. $A \cdot \bar{A} = 0$
5. $A + 0 = A$
6. $A + 1 = 1$
7. $A + \bar{A} = 1$
8. $A + A = A$

TEACHING WALLAH

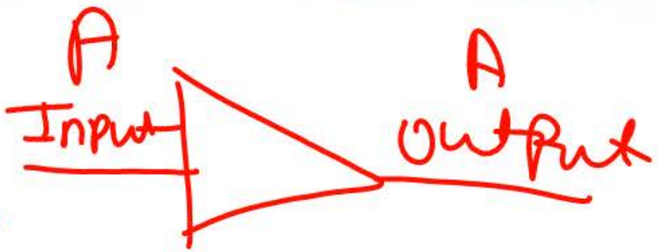
Conclusion

Boolean algebra laws are a series of laws and theorems that help understand why or how something happens with Boolean algebra, which is essential to any computer program.

The main goal of logic design is to simplify the logic as much as possible so that the final implementation is easy. To simplify the logic, you need to simplify the Boolean equations and expressions that represent the logic. Some laws and theorems have been proposed to simplify the Boolean equations. Using these laws and theorems makes it very easy to simplify or reduce the logical complexity of a Boolean expression or function.

TEACHING WALLAH

THANK YOU



Note:- In Buffer ckt
→ Always current Input = output
is pass in output
when Input is High